

RWorksheet_camasa#4c

April Marie

2025-01-04

```
library(ggplot2) library(dplyr)
```

#1. #1a. How to Import a CSV File

```
mpgdata <- read.csv("mpg.csv")
```

#b.

```
str(mpgdata)
```

```
## 'data.frame': 234 obs. of 12 variables:
## $ X : int 1 2 3 4 5 6 7 8 9 10 ...
## $ manufacturer: chr "audi" "audi" "audi" "audi" ...
## $ model : chr "a4" "a4" "a4" "a4" ...
## $ displ : num 1.8 1.8 2 2 2.8 2.8 3.1 1.8 1.8 2 ...
## $ year : int 1999 1999 2008 2008 1999 1999 2008 1999 1999 2008 ...
## $ cyl : int 4 4 4 4 6 6 6 4 4 4 ...
## $ trans : chr "auto(l5)" "manual(m5)" "manual(m6)" "auto(av)" ...
## $ drv : chr "f" "f" "f" "f" ...
## $ cty : int 18 21 20 21 16 18 18 16 20 ...
## $ hwy : int 29 29 31 30 26 26 27 26 25 28 ...
## $ fl : chr "p" "p" "p" "p" ...
## $ class : chr "compact" "compact" "compact" "compact" ...
```

2. Which manufacturer has the most models in this data set? Which model has the most variations?

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
manufacturer_count <- mpgdata %>%
  group_by(manufacturer) %>%
  summarise(model_count = n_distinct(model)) %>%
  arrange(desc(model_count))
most_models_manufacturer <- manufacturer_count[1, ]
model_variation_count <- mpgdata %>%
  group_by(model) %>%
  summarise(variation_count = n()) %>%
```

```

  arrange(desc(variation_count))
most_variations_model <- model_variation_count[1, ]
most_models_manufacturer

```

```

## # A tibble: 1 x 2
##   manufacturer model_count
##   <chr>             <int>
## 1 toyota             6

```

```
most_variations_model
```

```

## # A tibble: 1 x 2
##   model          variation_count
##   <chr>             <int>
## 1 caravan 2wd          11

```

A.

```

unique_models <- mpgdata %>%
  group_by(manufacturer) %>%
  summarise(unique_models = list(unique(model))) %>%
  arrange(manufacturer)
print(unique_models)

```

```

## # A tibble: 15 x 2
##   manufacturer unique_models
##   <chr>         <list>
## 1 audi         <chr [3]>
## 2 chevrolet    <chr [4]>
## 3 dodge        <chr [4]>
## 4 ford         <chr [4]>
## 5 honda        <chr [1]>
## 6 hyundai      <chr [2]>
## 7 jeep         <chr [1]>
## 8 land rover   <chr [1]>
## 9 lincoln      <chr [1]>
## 10 mercury     <chr [1]>
## 11 nissan       <chr [3]>
## 12 pontiac     <chr [1]>
## 13 subaru      <chr [2]>
## 14 toyota      <chr [6]>
## 15 volkswagen  <chr [4]>

```

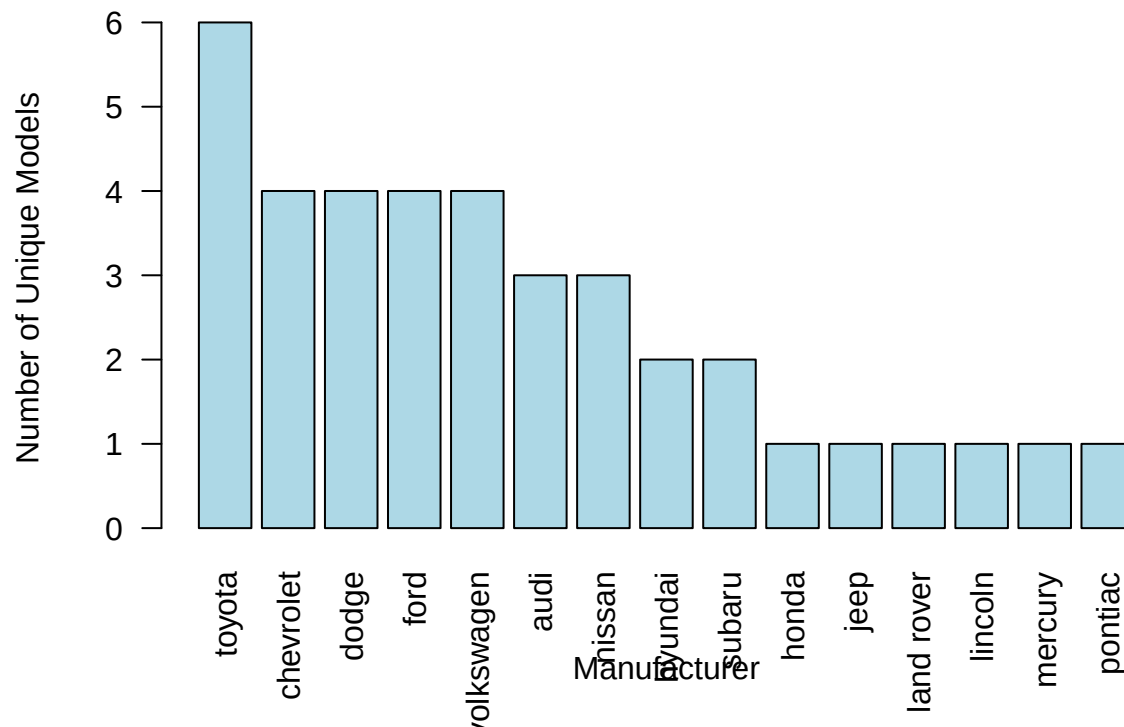
B.

```

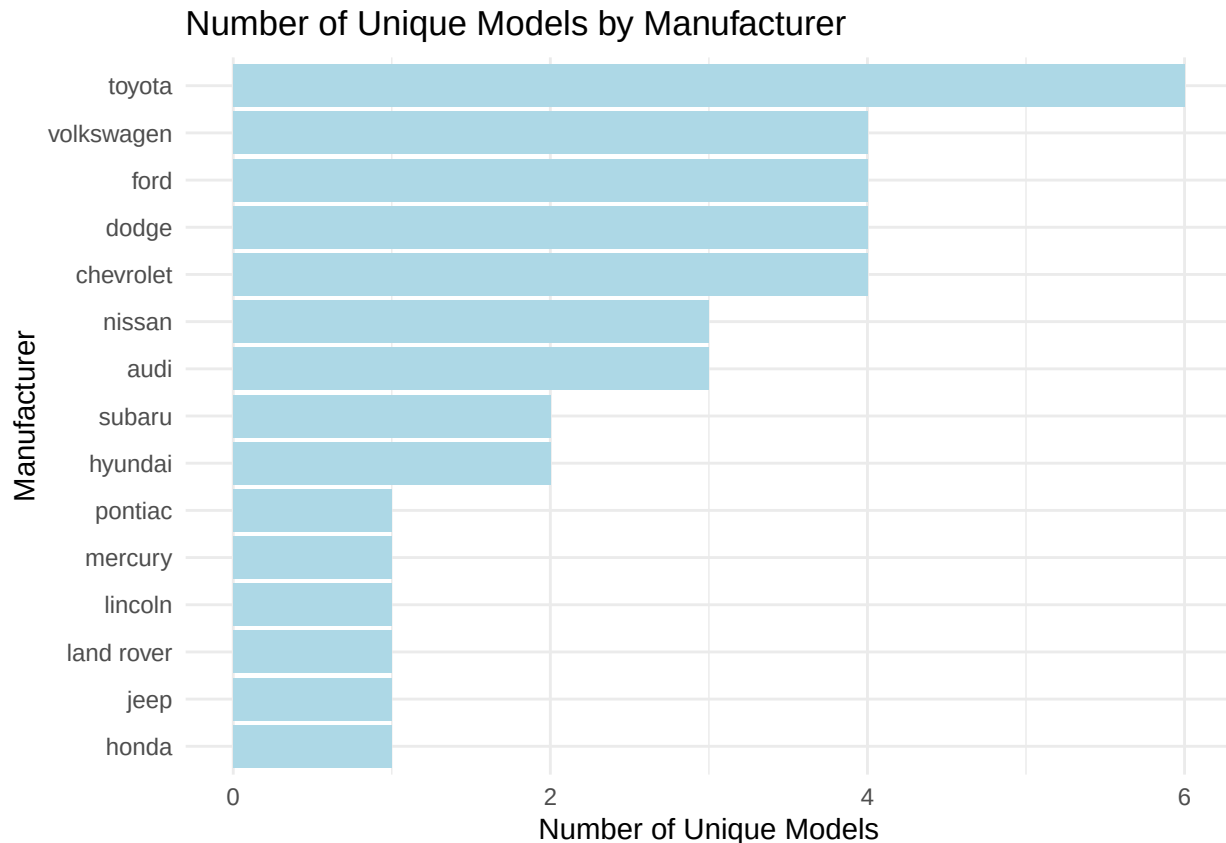
model_counts <- mpgdata %>%
  group_by(manufacturer) %>%
  summarise(unique_model_count = n_distinct(model)) %>%
  arrange(desc(unique_model_count))
barplot(model_counts$unique_model_count,
  names.arg = model_counts$manufacturer,
  las = 2,
  col = "lightblue",
  main = "Number of Unique Models by Manufacturer",
  xlab = "Manufacturer",
  ylab = "Number of Unique Models")
library(ggplot2)

```

Number of Unique Models by Manufacturer



```
ggplot(model_counts, aes(x = reorder(manufacturer, unique_model_count), y = unique_model_count)) +
  geom_bar(stat = "identity", fill = "lightblue") +
  coord_flip() +
  labs(title = "Number of Unique Models by Manufacturer",
       x = "Manufacturer",
       y = "Number of Unique Models") +
  theme_minimal()
```



2.

Same dataset will be used. You are going to show the relationship of the model and the manufacturer.

a. What does `ggplot(mpg, aes(model, manufacturer)) + geom_point()` show?

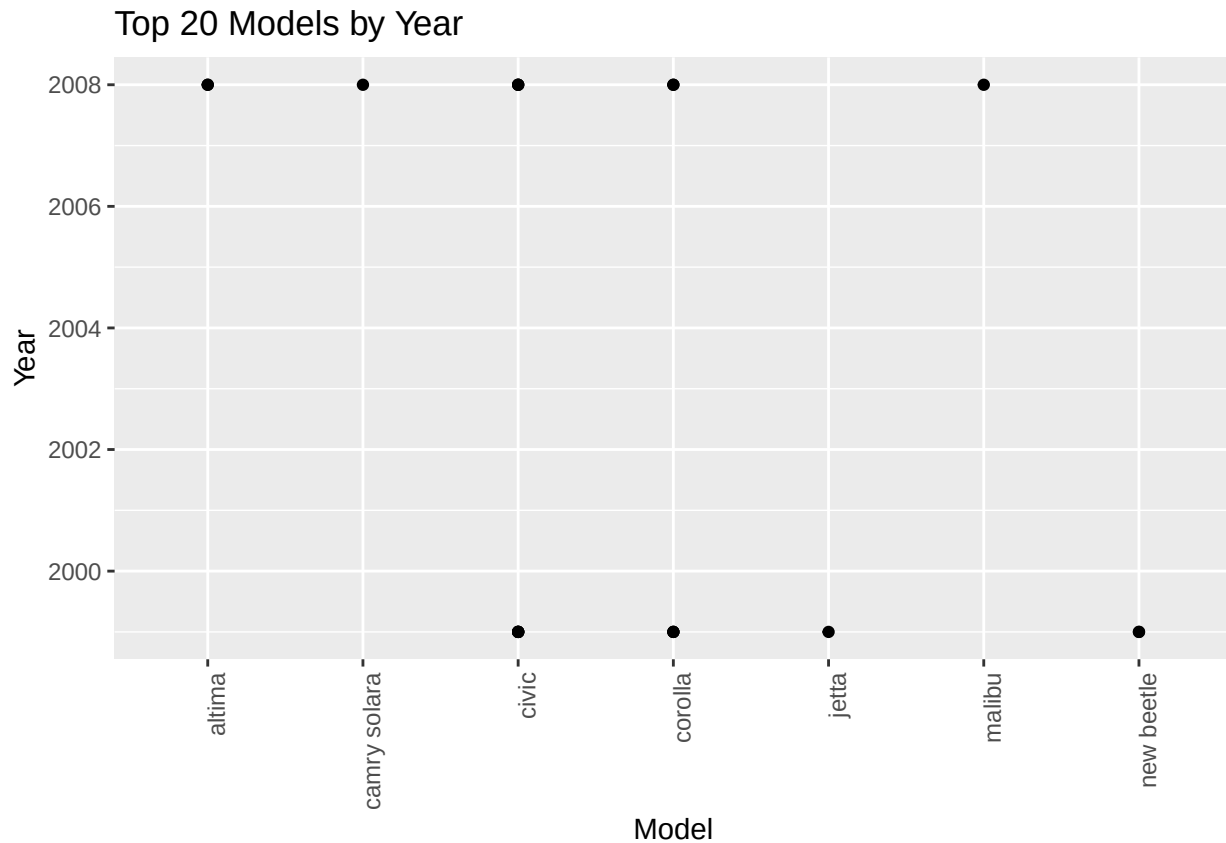
-the x-axis represents the model, and the y-axis represents the manufacturer. Since both variables are categorical, the plot will display points for each model-manufacturer pair. This can result in many overlapping points, making it difficult to identify any clear patterns or relationships. As a result, the plot may look cluttered and may not effectively convey the connection between these variables.

b. For you, is it useful? If not, how could you modify the data to make it more informative?

-As it is, the plot does not effectively reveal the relationship between the model and manufacturer due to the overplotting of points and the lack of numerical or continuous variables that could provide further insights.

3. Plot the model and the year using `ggplot()`. Use only the top 20 observations. Write the codes and its results.

```
library(ggplot2)
library(dplyr)
top_20_mpg <- mpgdata %>%
  arrange(desc(cty)) %>%
  head(20)
ggplot(top_20_mpg, aes(x = model, y = year)) +
  geom_point() +
  labs(title = "Top 20 Models by Year", x = "Model", y = "Year") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



4. Using the pipe (`%>%`), group the model and get the number of cars per model. Show codes and its result.

```
model_counts <- mpg %>%
  group_by(model) %>%
  summarise(number_of_cars = n()) %>%
  arrange(desc(number_of_cars))
print(model_counts)
```

```
## # A tibble: 38 x 2
##   model                number_of_cars
##   <chr>                  <int>
## 1 caravan 2wd             11
## 2 ram 1500 pickup 4wd     10
## 3 civic                   9
## 4 dakota pickup 4wd       9
## 5 jetta                   9
## 6 mustang                 9
## 7 a4 quattro              8
## 8 grand cherokee 4wd      8
## 9 impreza awd             8
## 10 a4                     7
## # i 28 more rows
```

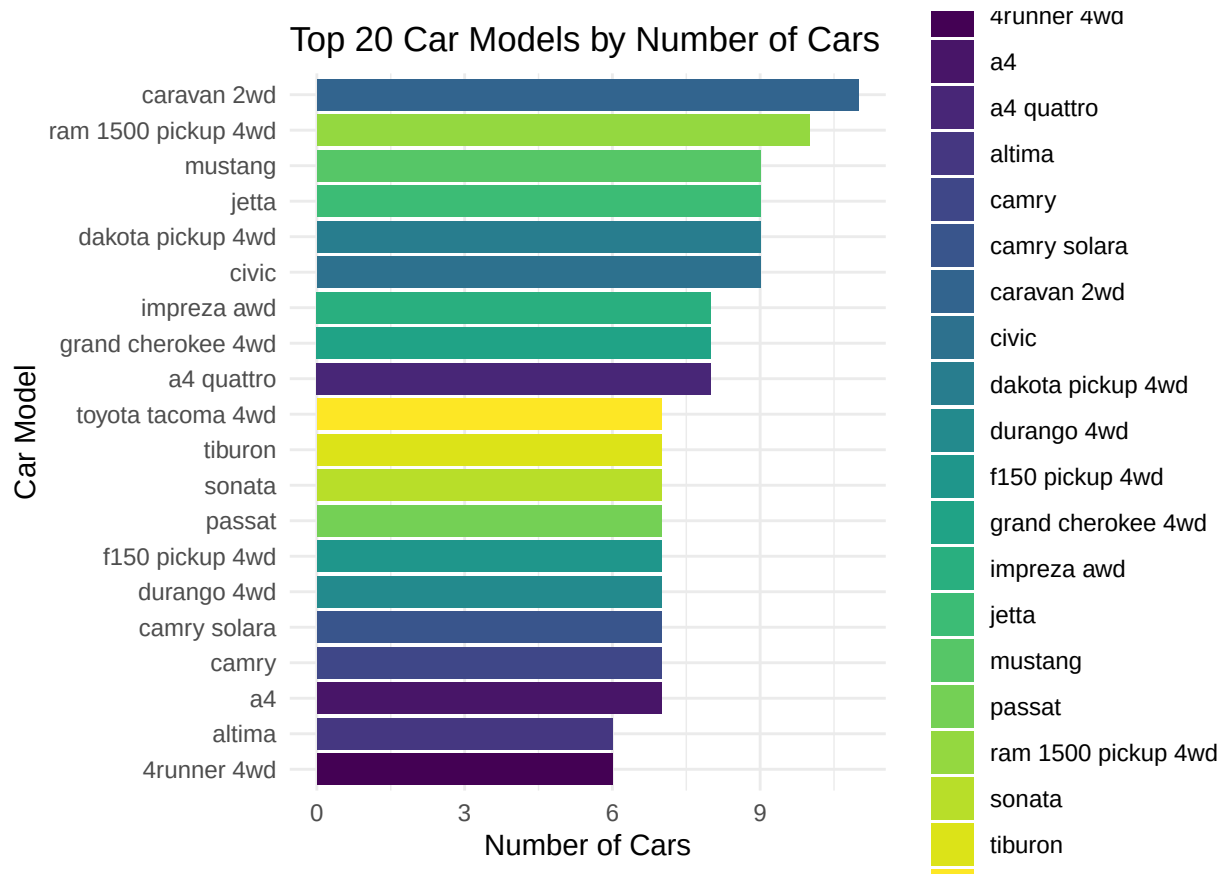
A.

```
top_models <- mpgdata %>%
  group_by(model) %>%
  summarise(number_of_cars = n()) %>%
```

```

arrange(desc(number_of_cars)) %>%
slice_head(n = 20)
ggplot(top_models, aes(x = reorder(model, number_of_cars), y = number_of_cars, fill = model)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(title = "Top 20 Car Models by Number of Cars",
       x = "Car Model",
       y = "Number of Cars",
       fill = "Model") +
  theme_minimal() +
  scale_fill_viridis_d()

```



b. Plot using the `geom_bar()` + `coord_flip()` just like what is shown below. Show codes and its result.

```

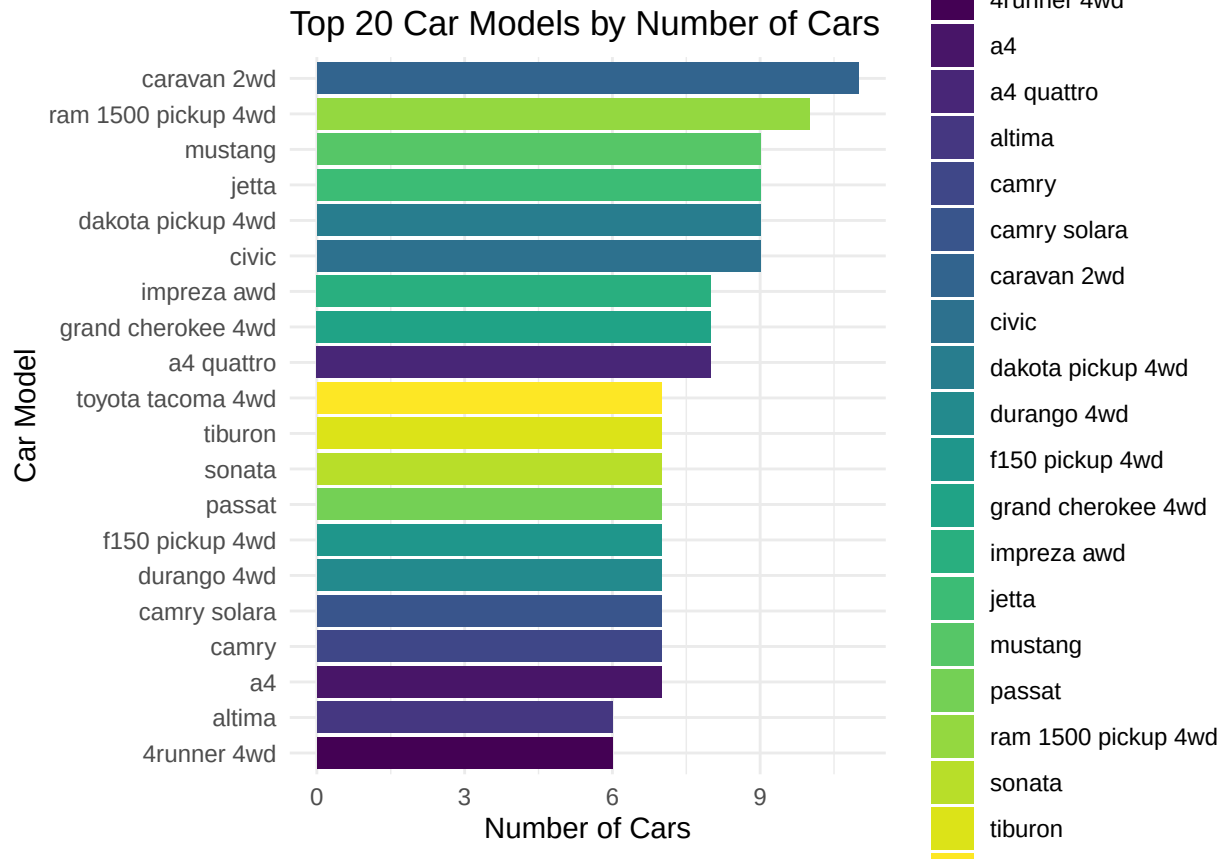
library(viridis)

## Loading required package: viridisLite

library(viridisLite)
top_models <- mpgdata %>%
  group_by(model) %>%
  summarise(number_of_cars = n()) %>%
  arrange(desc(number_of_cars)) %>%
  slice_head(n = 20)
ggplot(top_models, aes(x = reorder(model, number_of_cars), y = number_of_cars, fill = model)) +
  geom_bar(stat = "identity") +
  coord_flip() +

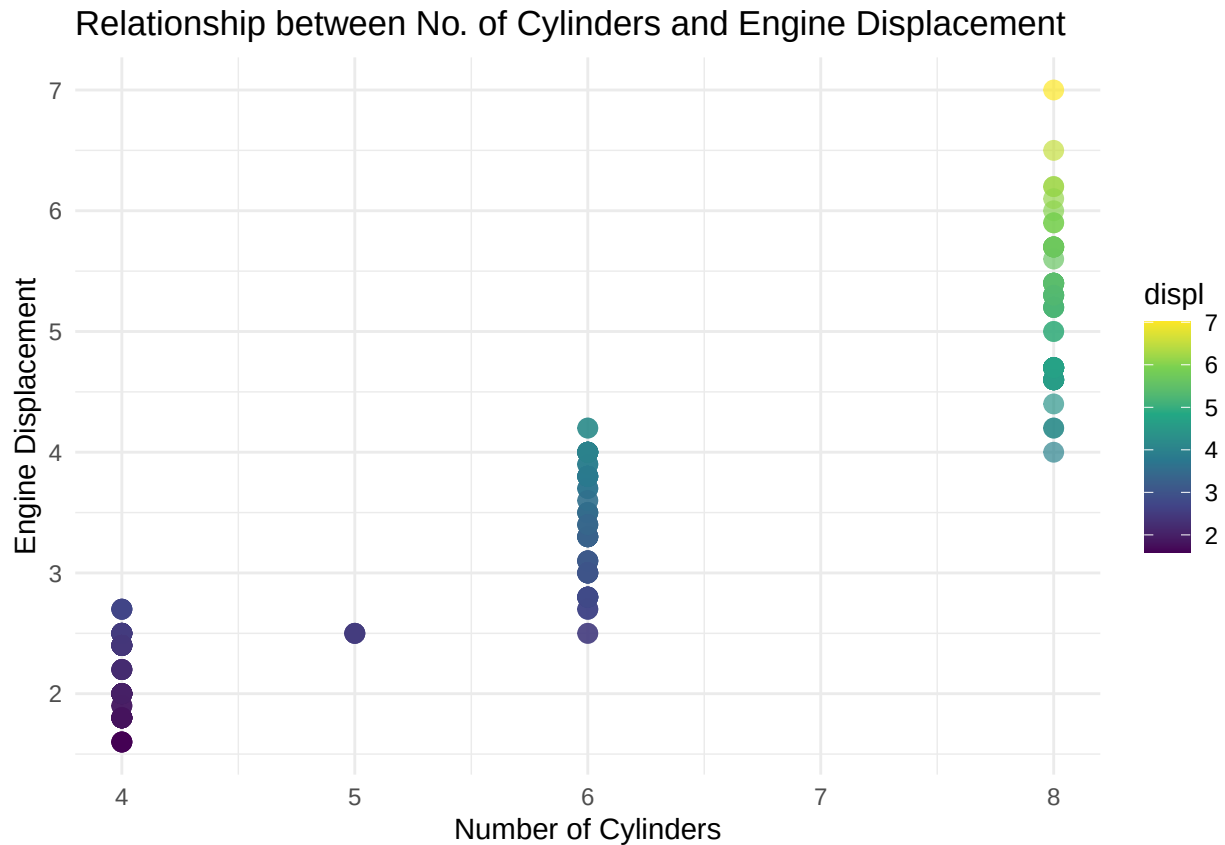
```

```
labs(title = "Top 20 Car Models by Number of Cars",
     x = "Car Model",
     y = "Number of Cars") +
theme_minimal() +
scale_fill_viridis_d()
```



5.

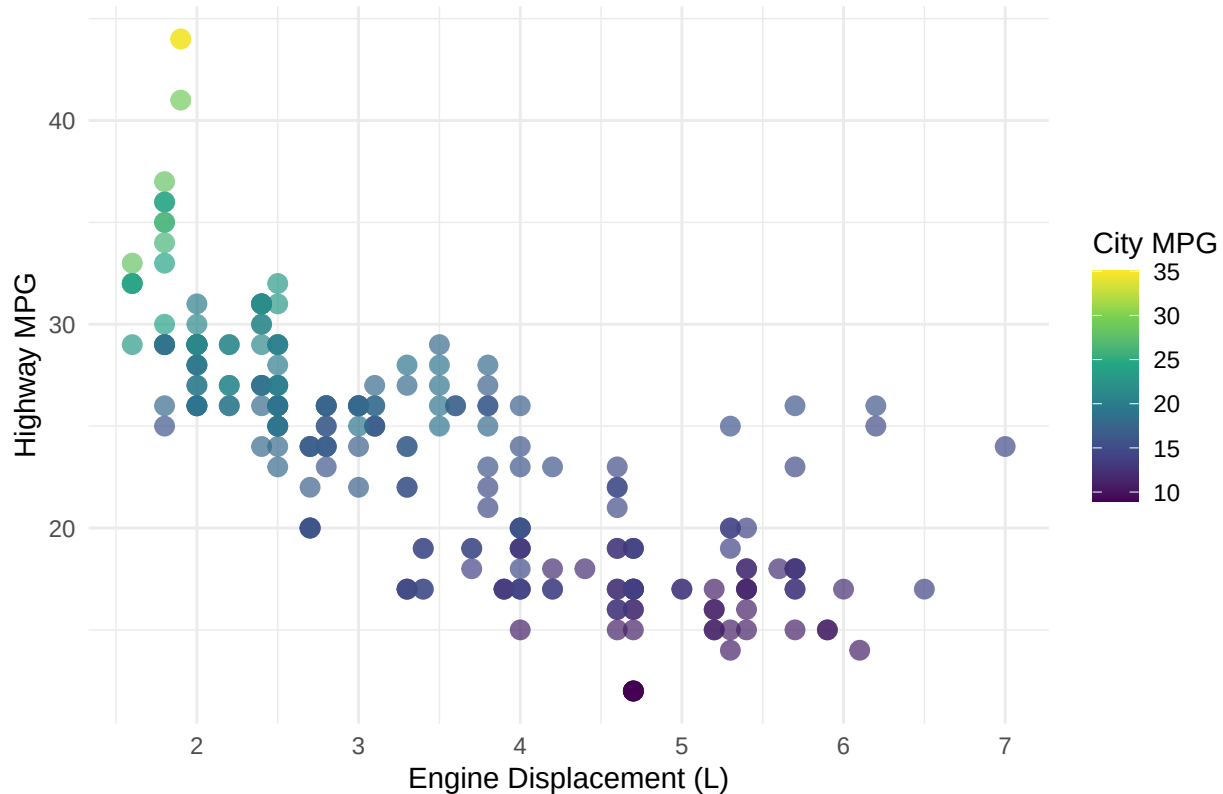
```
ggplot(mpgdata, aes(x = cyl, y = displ, color = displ)) +
geom_point(size = 3, alpha = 0.7) +
labs(title = "Relationship between No. of Cylinders and Engine Displacement",
     x = "Number of Cylinders",
     y = "Engine Displacement") +
theme_minimal() +
scale_color_viridis_c()
```



- a. How would you describe its relationship? Show the codes and its result. -The scatter plot shows a positive correlation between cylinder count and engine displacement, with higher cylinder counts generally linked to greater displacement. Clusters may form by cylinder count, and there can be variability, especially at higher counts. Outliers, such as high displacement with a low cylinder count, may suggest specialized engines.
6. Plot the relationship between displ (engine displacement) and hwy(highway miles per gallon). Mapped it with a continuous variable you have identified in #1-c. What is its result? Why it produced such output?

```
ggplot(mpg, aes(x = displ, y = hwy, color = cty)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Relationship between Engine Displacement and Highway MPG",
       x = "Engine Displacement (L)",
       y = "Highway MPG",
       color = "City MPG") +
  theme_minimal() +
  scale_color_viridis_c()
```


Relationship between Engine Displacement and Highway MPG



6.

Import the traffic.csv onto your R environment.

```
library(readr)
traffic_data <- read_csv("traffic.csv")

## Rows: 48120 Columns: 4
## -- Column specification -----
## Delimiter: ","
## dbl (3): Junction, Vehicles, ID
## dtm (1): DateTime
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
head(traffic_data)

## # A tibble: 6 x 4
##   DateTime          Junction Vehicles      ID
##   <dtm>            <dbl>    <dbl>    <dbl>
## 1 2015-11-01 00:00:00      1      15 20151101001
## 2 2015-11-01 01:00:00      1      13 20151101011
## 3 2015-11-01 02:00:00      1      10 20151101021
## 4 2015-11-01 03:00:00      1       7 20151101031
## 5 2015-11-01 04:00:00      1       9 20151101041
## 6 2015-11-01 05:00:00      1       6 20151101051
```

A. How many numbers of observation does it have? What are the variables of the traffic dataset.

```
num_observations <- nrow(traffic_data)
variables <- colnames(traffic_data)
```

```
num_observations
```

```
## [1] 48120
```

```
variables
```

```
## [1] "DateTime" "Junction" "Vehicles" "ID"
```

B. subset the traffic dataset into junctions. What is the R codes and its output?

```
junctions_data <- subset(traffic_data, Junction == TRUE)
head(junctions_data)
```

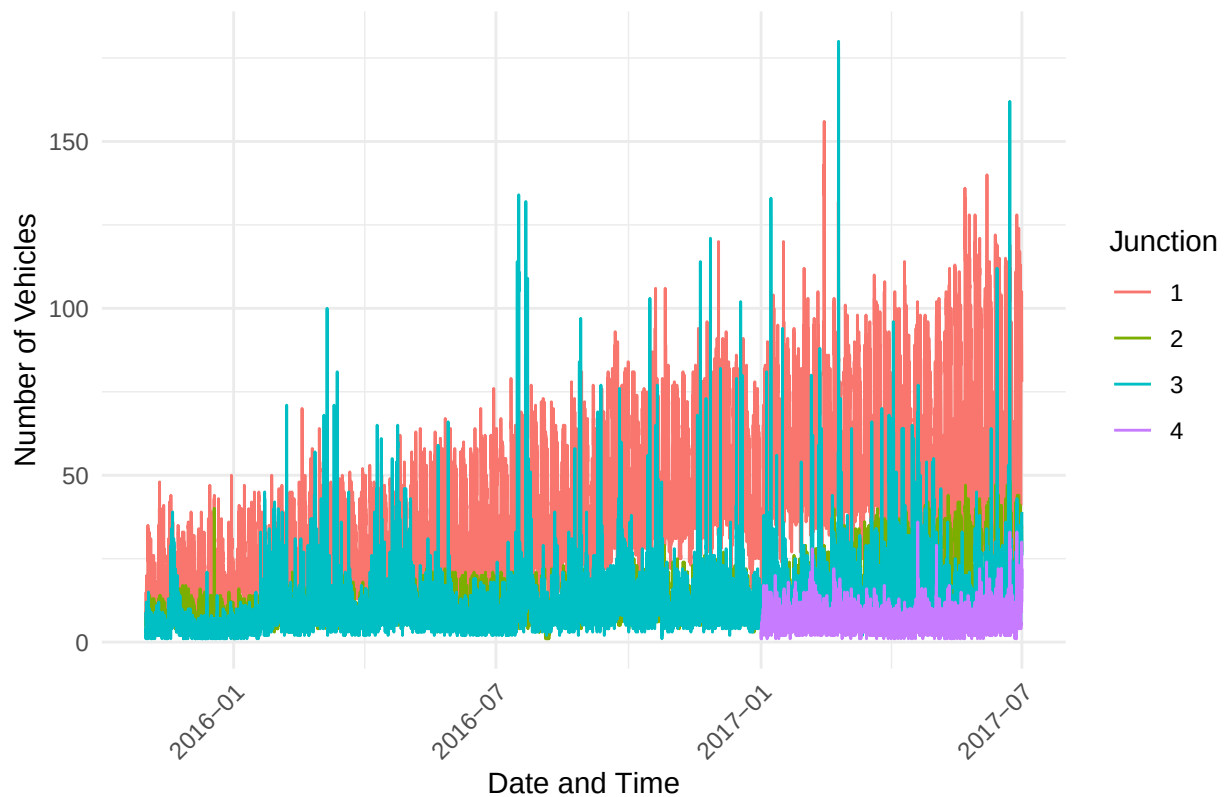
```
## # A tibble: 6 x 4
```

```
##   DateTime          Junction Vehicles      ID
##   <dtm>             <dbl>    <dbl>    <dbl>
## 1 2015-11-01 00:00:00         1      15 20151101001
## 2 2015-11-01 01:00:00         1      13 20151101011
## 3 2015-11-01 02:00:00         1      10 20151101021
## 4 2015-11-01 03:00:00         1       7 20151101031
## 5 2015-11-01 04:00:00         1       9 20151101041
## 6 2015-11-01 05:00:00         1       6 20151101051
```

C. Plot each junction in a using `geom_line()`. Show your solution and output.

```
library(ggplot2)
ggplot(traffic_data, aes(x = DateTime, y = Vehicles, color = factor(Junction))) +
  geom_line() +
  labs(title = "Vehicle Counts at Junctions Over Time",
       x = "Date and Time",
       y = "Number of Vehicles",
       color = "Junction") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Vehicle Counts at Junctions Over Time



7.

A. How many observations does alexa_file has? What about the number of columns? Show your solution and answer.

```
library(readxl)
alexa_data <- read_excel("alexa_file.xlsx")
dimensions <- dim(alexa_data)
num_rows <- dimensions[1]
num_columns <- dimensions[2]
num_rows
```

```
## [1] 3150
```

```
num_columns
```

```
## [1] 5
```

B. group the variations and get the total of each variations. Use dplyr package. Show solution and answer.

```
library(dplyr)
variation_totals <- alexa_data %>%
  group_by(variation) %>%
  summarise(total = n())
variation_totals
```

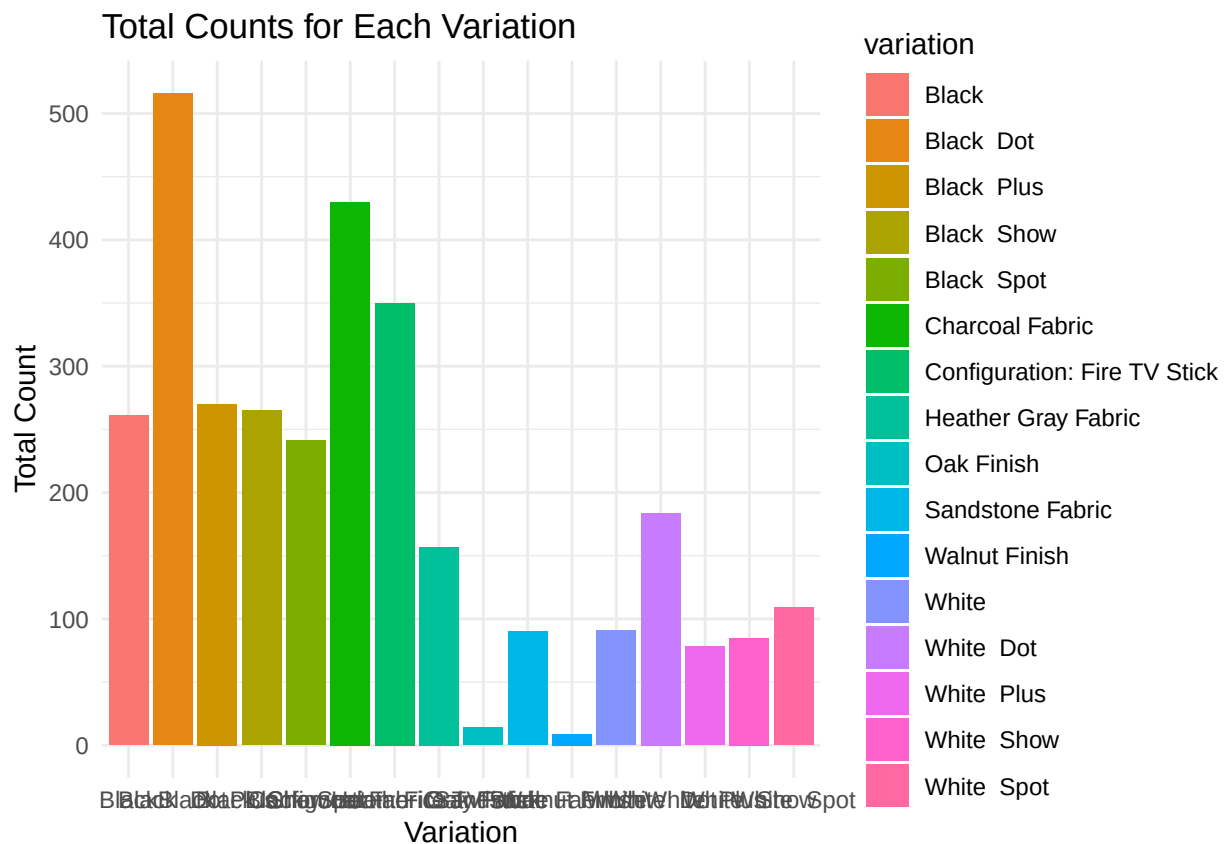
```
## # A tibble: 16 x 2
```

```
##   variation      total
##   <chr>      <int>
## 1 Black      261
```

```
## 2 Black Dot 516
## 3 Black Plus 270
## 4 Black Show 265
## 5 Black Spot 241
## 6 Charcoal Fabric 430
## 7 Configuration: Fire TV Stick 350
## 8 Heather Gray Fabric 157
## 9 Oak Finish 14
## 10 Sandstone Fabric 90
## 11 Walnut Finish 9
## 12 White 91
## 13 White Dot 184
## 14 White Plus 78
## 15 White Show 85
## 16 White Spot 109
```

C. Plot the variations using the `ggplot()` function. What did you observe? Complete the details of the graph.

```
library(ggplot2)
ggplot(variation_totals, aes(x = variation, y = total, fill = variation)) +
  geom_bar(stat = "identity") +
  labs(title = "Total Counts for Each Variation", x = "Variation", y = "Total Count") +
  theme_minimal()
```

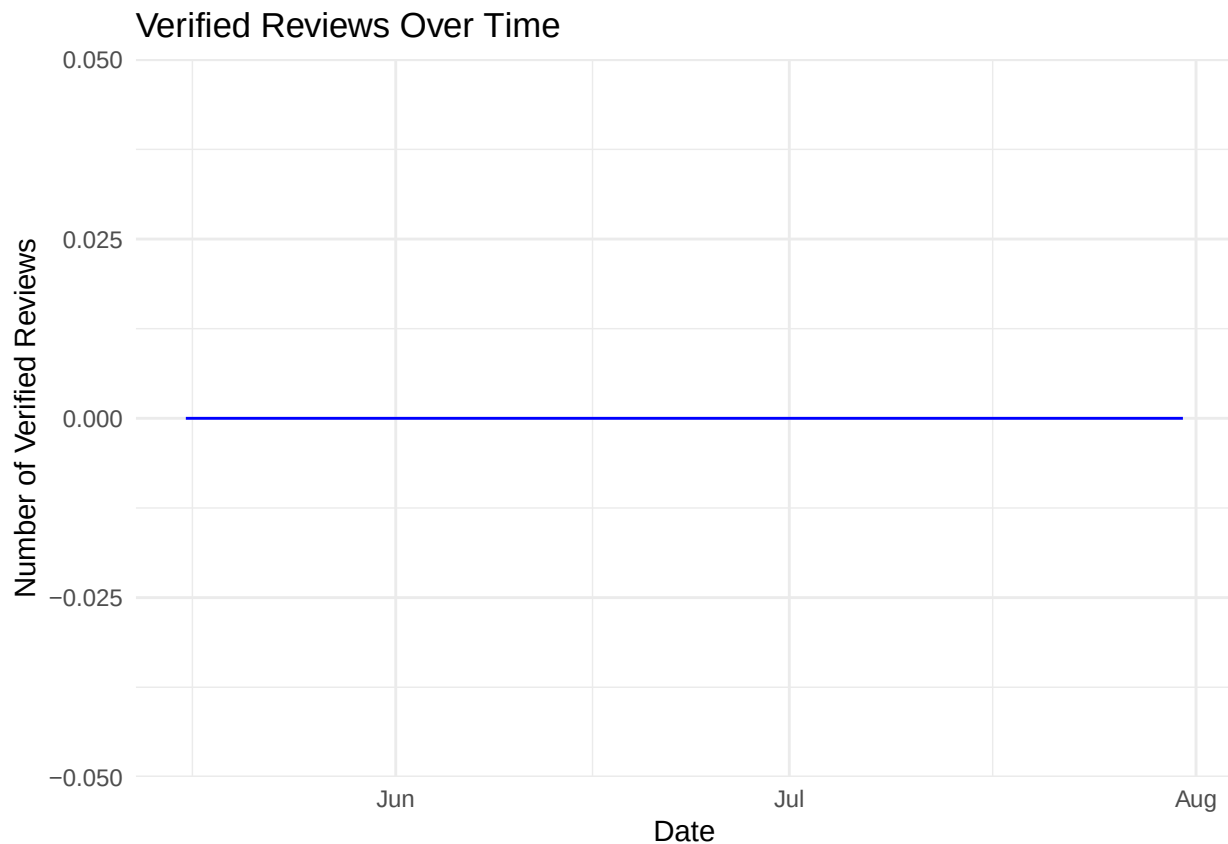


D. Plot a `geom_line()` with the date and the number of verified reviews. Complete the details of the graphs.

```
alexa_data$verified_reviews <- as.numeric(alexa_data$verified_reviews)
```

```
## Warning: NAs introduced by coercion
```

```
reviews_by_date <- alexa_data %>%
  group_by(date) %>%
  summarise(total_verified_reviews = sum(verified_reviews, na.rm = TRUE))
library(ggplot2)
ggplot(reviews_by_date, aes(x = date, y = total_verified_reviews)) +
  geom_line(color = "blue") +
  labs(title = "Verified Reviews Over Time", x = "Date", y = "Number of Verified Reviews") +
  theme_minimal()
```



E. Get the relationship of variations and ratings. Which variations got the most highest in rating?
Plot a graph to show its relationship. Show your solution and answer.

```
variation_ratings <- alexa_data %>%
  group_by(variation) %>%
  summarise(average_rating = mean(rating, na.rm = TRUE))
ggplot(variation_ratings, aes(x = variation, y = average_rating, fill = variation)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Rating by Variation", x = "Variation", y = "Average Rating") +
  theme_minimal()
```

